Replication Plan

Mathematical Foundations of the GraphBLAS

OSTI ID: 1379592

Auto-generated Replication Blueprint

April 8, 2026

Contents

Paper Overview

This paper by Kepner et al. presents the mathematical foundations underlying the GraphBLAS standard—a building-block approach to graph algorithms expressed via linear-algebraic operations on generalized semirings. The work defines generalized matrix and vector operations (e.g., matrix multiply on arbitrary semirings, element-wise operations, masking, extraction, assignment) and demonstrates how classical graph algorithms (BFS, shortest paths, centrality, triangle counting, etc.) map onto these primitives.

The paper is primarily *definitional/algorithmic*: it specifies algebraic structures (monoids, semirings), generalized matrix operations, and gives small worked examples. Replication therefore focuses on implementing the operations, verifying them on the paper's examples, and demonstrating the graph algorithms.

Detailed Workflow

Phase 1: Algebraic Foundations

1.1 Define algebraic structures.

- Implement a `Monoid` class with an associative binary operator and identity element.
- Implement a `Semiring` class combining an additive monoid and a multiplicative monoid satisfying distributivity; include an annihilator (zero) element.
- Provide built-in instances: arithmetic semiring $(R, +, \times, 0, 1)$, tropical (min-plus) semiring $(R \cup \{+\infty\}, \min, +, +\infty, 0)$, Boolean semiring $(\{true, false\}, \vee, \wedge, false, true)$, and others referenced in the paper.

1.2 Define sparse matrix/vector representations.

- Represent matrices as dictionaries of (row, col) $\rightarrow$ value or via SciPy sparse formats.
- Support an explicit "no-value" (structural zero) distinct from the additive identity when needed.

Phase 2: Core GraphBLAS Operations

2.1 Matrix–matrix multiply (`mxm`). Generalized $\mathbf{C} = \mathbf{A} \oplus . \otimes \mathbf{B}$ over an arbitrary semiring.

2.2 Matrix–vector multiply (`mxv`) and vector–matrix multiply (`vxm`).

2.3 Element-wise operations (`eWiseAdd`, `eWiseMult`). Union and intersection semantics on the structure of two matrices/vectors.

2.4 Extract and Assign sub-matrices/sub-vectors using index arrays.

2.5 Apply a unary function element-wise to a matrix or vector.

2.6 Reduce a matrix to a vector (row-wise or column-wise) or a vector to a scalar using a monoid.

2.7 Transpose operation.

2.8 Masking and accumulation: structural and value-based masks; accumulator semantics.

2.9 Kronecker product over a semiring.

Phase 3: Verification with Paper Examples

3.1 Reproduce every small numeric example in the paper (e.g., the 4×4 adjacency-matrix BFS example, the shortest-path tropical semiring example, the triangle-counting example).

3.2 Compare outputs element-by-element against the paper's stated results.

Phase 4: Graph Algorithm Demonstrations

4.1 **Breadth-First Search (BFS)** via repeated matrix–vector multiplication on the Boolean semiring.

4.2 **Single-Source Shortest Paths** via repeated multiplication on the tropical semiring.

4.3 **Triangle counting** via masked matrix multiply and reduce.

4.4 **Betweenness centrality** via the BC algorithm expressed in GraphBLAS primitives.

4.5 **PageRank / k-Truss / connected components** if time permits, as extended demonstrations.

Phase 5: Performance and Scalability (Optional)

5.1 Benchmark the Python/SciPy prototype against the reference SuiteSparse:GraphBLAS C implementation on standard graph datasets (e.g., SNAP graphs, Graph500 RMAT).

5.2 Report timings and memory usage.

Datasets

The paper is primarily definitional and uses small, synthetic, hand-constructed matrices. No external datasets are required for core replication.

Dataset / Source	Type	Location / Notes
Paper's inline examples (Tables/Figures)	Synthetic small matrices	Extracted directly from the paper text
SNAP graph datasets (optional benchmarking)	Real-world graphs	`https://snap.stanford.edu/data/`
SuiteSparse Matrix Collection (optional)	Sparse matrices	`https://sparse.tamu.edu/`
Graph500 RMAT generator (optional)	Synthetic power-law graphs	`https://graph500.org/`

Models, Tools, and Software

Programming Languages

- **Python 3.8+** — primary implementation language for the prototype.
- **C/C++** — only needed if benchmarking against SuiteSparse:GraphBLAS.

Libraries and Frameworks

Tool / Library	Version	Purpose / URL
NumPy	≥ 1.21	Dense array operations; semiring arithmetic
SciPy (scipy.sparse)	≥ 1.7	Sparse matrix storage and basic linear algebra
python-graphblas	latest	Python wrapper for SuiteSparse:GraphBLAS; provides high-level GraphBLAS API `https://github.com/python-graphblas/python-graphblas`
SuiteSparse:GraphBLAS	≥ 7.0	Reference C implementation of the GraphBLAS spec `https://github.com/DrTimothyAldenDavis/GraphBLAS`
NetworkX	≥ 2.6	Graph construction, visualization, and validation of algorithm outputs
Matplotlib	≥ 3.4	Plotting and visualization
pytest	≥ 6.0	Unit testing framework for verifying operations

Hardware Requirements

- For the paper's small examples: any modern laptop suffices.
- For optional large-scale benchmarking: multi-core workstation with ≥ 32 GB RAM recommended.

Reference Documents

- The GraphBLAS C API Specification: `https://graphblas.org/docs/GraphBLAS_API_C_v2.0.0.pdf`
- GraphBLAS Forum: `https://graphblas.org/`